

Reference Links:

Research Paper: Framework GNN-AID: graph Neural Network Analysis Interpretation and Defense. Link: <https://arxiv.org/abs/2505.03424v1>

GitHub Repository: <https://github.com/ispras/GNN-AID/>

GNN Basics: <https://www.geeksforgeeks.org/deep-learning/what-are-graph-neural-networks/>

This project is about analyzing and protecting Graph Neural Networks (GNNs) using a powerful tool called GNN-AID, which is already an open-source project.

What are Graph Neural Networks (GNNs)?

- GNNs are special neural networks used to process graph data (like social networks, road maps, citation networks).
- They are useful in tasks like fraud detection, social recommendation, molecule analysis, etc.

Why is Defense needed?

- GNNs can be attacked by hackers using “adversarial attacks” — these are smart changes to the graph that fool the model into making wrong predictions.
- Your project focuses on both understanding and defending against these attacks.

ProjectScope:

GNN-AID is an open-source Python framework and interactive web platform for Graph Neural Network Analysis, Interpretation, and Defense.

Built on PyTorch-Geometric, it provides:

***Comprehensive Trust Toolkit:**

integrates 8 post-hoc and self-interpretable explanation methods,
7 attack algorithms (e.g., Netstack, PGD, CLGA),
and 7 defense strategies (e.g., Adversarial Training, Gradient Regularization, GNNGuard) in a single API

***No-Code Web Interface:**

offers dataset/model selection,
real-time graph visualizations,
an interactive model builder,
and interpretation/attack-defense workflows all via GUI .

***MLOps Support:**

includes experiment reproducibility and result versioning to streamline research and deployment of trusted GNNs.

What is GNN-AID?

- It's a toolkit + web interface built using Python and PyTorch-Geometric, with features like:
- Visualizing how GNNs make decisions.
- Running different types of attacks on GNNs.
- Trying out defense methods (like adversarial training).
- Using a No-Code interface to make it beginner-friendly.
- Tracking experiments and versions using MLOps tools.

Need of GNN-AID:

While interpretability and robustness are crucial for Trusted AI, most existing frameworks:

- Focus on non-graph data (images, text, tabular)
 - Rarely integrate both interpretability and robustness tools for GNNs in one package
 - Lack a user-friendly interface for non-expert users
- If a beginner just wants to see how adding noise to a graph changes the predictions, in most frameworks they must:
1. Load data via code
 2. Write model training code
 3. Write the attack script
 4. Write visualization code

In GNN-AID, they can just use the web interface, click on an attack, see the graph visualization, and check the model's performance — no coding required.

- Do not offer a comprehensive set of attacks, defenses, and interpretation methods tailored to graph data.

GNN-AID aims to fill this gap by providing a unified, extensible, and accessible platform specifically for GNNs, allowing for integrated analysis, interpretation, attack simulation, and defense evaluation — all within a single API and GUI.

Our Tasks:

Should let users explore how GNNs work.

Test and defend against attacks

Helps researchers by simplifying experiments and results tracking.

KeyPoints to understand in Research Paper:

- Understand how GNNs work - GCN, GAT
- Types of attacks and defenses exist in the framework.
- How interpretability is handled - LIME, SHAP
- How the web UI+ MLOps components work.

Problem Statement in detail:

GNN-AID is an open-source Python framework and interactive web platform for Graph Neural Network Analysis, Interpretation, and Defense.”

Means: It's both a Python library (so you can code with it) and a web UI (so you can use it without coding).

It focuses on three main areas:

1. **Analysis** → train and evaluate GNNs, study their behavior.
2. **Interpretation** → explain how GNNs make decisions (like “which nodes/edges were important for this prediction?”).
3. **Defense** → protect GNNs against adversarial attacks (fake or modified graph data designed to fool the model).

“Built on **PyTorch-Geometric...**”

PyTorch-Geometric (PyG) is a popular deep learning library for working with graphs in PyTorch.

GNN-AID reuses PyG for core GNN tasks but adds tools on top for explanations, attacks, defenses, and visualization.

core GNN tasks usually include:

1. Graph Data Handling

Loading datasets (e.g., Cora, Citeseer, PubMed, MUTAG).

Representing graphs as adjacency matrices / edge lists, node features, and labels.

Preprocessing: normalization of features, train/val/test splits.

2. Model Construction

Defining graph layers (GCNConv, GraphSAGEConv, GATConv, GINConv, etc.).

Stacking layers to build GNNs for different tasks.

Using built-in PyG modules for pooling, readout, and graph-level embeddings.

3. Training Loop

Forward pass → propagate messages across graph layers.

Loss computation (cross-entropy for classification, etc.).

Backward pass & optimization using PyTorch (Adam, SGD).

Evaluation: accuracy, F1, AUROC, etc.

4. Prediction Tasks

The three most common ones that GNN-AID (via PyG) supports are:

Node Classification → predict a label for each node (e.g., topic classification in citation networks).

Link Prediction → predict if an edge exists between two nodes (e.g., recommend a friend in a social network).

Graph Classification → predict a label for an entire graph (e.g., classify molecules as toxic or non-toxic).

5. Utilities

Mini-batching large graphs.

GPU acceleration.

Data transforms (edge dropout, normalization, augmentation).

Hooks for tracking activations/gradients (useful for interpretability).

To summarize,

- PyG handles the heavy lifting (datasets, GNN layers, training, prediction).
 - GNN-AID adds the “trust” layer → attacks, defenses, explainers, visualization, experiment tracking.
-

Key Features:

1. Comprehensive Trust Toolkit

- 8 Explanation Methods → These are algorithms to interpret GNN predictions (like GNNExplainer, PGExplainer, SubgraphX, etc.). Some are post-hoc (analyze after training), others are self-interpretable (built into the model).
- 7 Attack Algorithms → Different adversarial methods that break GNNs, such as:
 - Nettack (modifies graph structure),
 - PGD (gradient-based feature/structure attack),
 - CLGA (poisoning attack using meta-learning).
- 7 Defense Strategies → Countermeasures to make GNNs more robust, e.g.:
 - Adversarial Training (train with adversarial examples),

- Gradient Regularization (stabilizes training),
- GNNGuard (detects/removes suspicious connections).
- Single API → All of this is accessible with one consistent programming interface.

2. No-Code Web Interface

- You don't need to be a coder to try it. The web UI lets you:
 - Pick datasets (like Cora, Citeseer).
 - Select or build models interactively.
 - Run attacks/defenses with clicks instead of code.
 - See real-time graph visualizations (nodes, edges, embeddings, explanations).

Basically: a “sandbox” to experiment visually.

3. MLOps Support

- Reproducibility → ensures that if you run the same experiment later, you'll get the same results (important in research).
- Result Versioning → keeps track of multiple experiments, models, attacks, and defenses (like Git for experiments).
- Helps researchers and developers manage experiments efficiently and compare results easily.

What is MLOps?

- MLOps = Machine Learning Operations
- It is a set of practices, tools, and workflows that make it easier to develop, train, deploy, monitor, and maintain machine learning models efficiently — similar to how DevOps does it for software development.
- Think of it like:

“MLOps = DevOps + Machine Learning”
- It focuses on automation, reproducibility, scalability, and collaboration in ML projects.

Why MLOps is Needed?

- Without MLOps, ML projects often face problems like:
- Models work on your laptop but fail in production.
- Reproducing results from past experiments becomes hard.
- Difficult to manage multiple models, datasets, and versions.
- Hard to monitor model performance drift over time.
- For frameworks like GNN-AID, where you run different attacks, defenses, explainers, and multiple experiments, MLOps tools help you keep everything organized and reproducible.

How GNN-AID Uses MLOps?

- Experiment reproducibility → Run the same attack-defense-explainer pipeline later, get identical results.
- Result versioning → Keeps track of different experiments automatically.
- Organized workflows → Makes it easier for researchers to compare different models and explainers without confusion.

MLOps in GNN-AID= Manage experiments, models, datasets, and results in an organized, reproducible, and automated way

Simply put,

Imagine you're running 20 experiments on GNNs with different:

Models (GCN, GAT, GraphSAGE)

Attacks (PGD, Nettack, CLGA)

Defenses (AdvTrain, GNNGuard, Jaccard)

- Without MLOps → You'll lose track of what worked and what didn't.
- With MLOps → Every experiment, result, and model version is automatically logged and reproducible.

Pipeline in the project:

Datasets → GNN Models → Attacks → Defenses → Explanations → MLOps Layer → Web Interface

So, in plain words:

GNN-AID is a one-stop platform that lets you train graph neural networks, test how they can be attacked, defend them, and explain their predictions — all either by writing code (Python + PyTorch-Geometric) or via a user-friendly web app.

It's designed to make trustworthy AI for graphs easier to study, prototype, and deploy, especially for researchers and students who don't want to re-implement every attack/defense/explainer from scratch.
